

DocChat API v2.1

Technical Documentation

Version: 2.1.3

Last Updated: September 20, 2025

Base URL: <https://api.docchat.com/v2>

Table of Contents

1. [Getting Started](#)
 2. [Authentication](#)
 3. [API Endpoints](#)
 4. [Error Handling](#)
 5. [Rate Limiting & Security](#)
 6. [SDK Installation](#)
 7. [Webhooks](#)
 8. [Code Examples](#)
-

Getting Started

The DocChat API allows you to integrate intelligent document processing and chat capabilities into your applications. This RESTful API supports document upload, processing, and retrieval of AI-generated insights.

Key Features:

- Document upload and processing (PDF, DOCX, TXT)
- RAG (Retrieval-Augmented Generation) chat interface
- Real-time processing status updates
- Webhook notifications for async operations

Prerequisites:

- API key (obtain from [DocChat Dashboard](#))
 - HTTPS-only requests
 - JSON content type for POST/PUT requests
-

Authentication

All API requests require authentication using your API key in the request header.

API Key Authentication

```
Authorization: Bearer dc_live_1234567890abcdef
Content-Type: application/json
```

Getting Your API Key

1. Sign up at [DocChat Dashboard](#)
2. Navigate to **Settings > API Keys**
3. Generate a new key for your environment:
 - o dc_test_* for development/testing
 - o dc_live_* for production

Security Note: Never expose API keys in client-side code. Store them securely as environment variables.

API Endpoints

Documents

Upload Document

Upload and process a document for chat interaction.

Endpoint: POST /documents

Request Headers:

```
Authorization: Bearer dc_live_1234567890abcdef
Content-Type: multipart/form-data
```

Request Body:

```
// Form data
{
  "file": [binary file data],
  "name": "Q3_Financial_Report.pdf",
  "tags": ["financial", "quarterly"],
  "processing_mode": "standard" // or "fast", "comprehensive"
}
```

Response (201 Created):

```
{
  "document_id": "doc_2Nq8x7Hn9mKpL3R5",
  "name": "Q3_Financial_Report.pdf",
  "size_bytes": 2458624,
  "status": "processing",
  "upload_time": "2025-09-20T14:30:22Z",
  "processing_mode": "standard",
  "estimated_completion": "2025-09-20T14:35:22Z",
  "tags": ["financial", "quarterly"],
  "metadata": {
    "pages": 47,
    "file_type": "pdf",
    "language": "en"
  }
}
```

Get Document Status

Check processing status of an uploaded document.

Endpoint: GET /documents/{document_id}

Response (200 OK):

```
{
  "document_id": "doc_2Nq8x7Hn9mKpL3R5",
  "name": "Q3_Financial_Report.pdf",
  "status": "completed",
  "processing_progress": 100,
  "upload_time": "2025-09-20T14:30:22Z",
  "completion_time": "2025-09-20T14:34:18Z",
  "analysis_results": {
    "confidence_score": 0.94,
    "entities_extracted": 156,
    "topics_identified": 12,
    "summary": "Financial report showing Q3 revenue growth of 23%..."
  }
}
```

List Documents

Retrieve all documents for your account.

Endpoint: GET /documents

Query Parameters:

- limit: Number of results (default: 50, max: 100)
- offset: Pagination offset (default: 0)

- status: Filter by status (processing, completed, failed)
- tags: Filter by tags (comma-separated)

Response (200 OK):

```
{
  "documents": [
    {
      "document_id": "doc_2Nq8x7Hn9mKpL3R5",
      "name": "Q3_Financial_Report.pdf",
      "status": "completed",
      "upload_time": "2025-09-20T14:30:22Z",
      "tags": ["financial", "quarterly"]
    }
  ],
  "total_count": 1,
  "has_more": false
}
```

Chat

Create Chat Session

Start a new chat session with processed documents.

Endpoint: POST /chat/sessions

Request Body:

```
{
  "document_ids": ["doc_2Nq8x7Hn9mKpL3R5"],
  "session_name": "Q3 Financial Analysis",
  "settings": {
    "temperature": 0.7,
    "max_tokens": 1000,
    "search_mode": "semantic"
  }
}
```

Response (201 Created):

```
{
  "session_id": "chat_8Kf2x9Qm3nPpL7R1",
  "session_name": "Q3 Financial Analysis",
  "created_at": "2025-09-20T15:00:00Z",
  "document_count": 1,
  "status": "active"
}
```

Send Message

Send a question to the chat session.

Endpoint: POST /chat/sessions/{session_id}/messages

Request Body:

```
{
  "message": "What was the revenue growth percentage in Q3?",
  "include_sources": true,
  "max_response_length": 500
}
```

Response (200 OK):

```
{
  "message_id": "msg_3Mp9x2Fn8kLpQ4R6",
  "response": "Based on the Q3 Financial Report, the revenue growth was 23% compared to the previous quarter. This growth was primarily driven by increased sales in the cloud services division.",
  "confidence": 0.92,
  "response_time_ms": 1247,
  "sources": [
    {
      "document_id": "doc_2Nq8x7Hn9mKpL3R5",
      "page": 3,
      "excerpt": "Q3 revenue increased by 23% to $127.3 million...",
      "relevance_score": 0.94
    }
  ],
  "timestamp": "2025-09-20T15:05:30Z"
}
```

Get Chat History

Retrieve conversation history for a session.

Endpoint: GET /chat/sessions/{session_id}/messages

Response (200 OK):

```
{
  "session_id": "chat_8Kf2x9Qm3nPpL7R1",
  "messages": [
    {
      "message_id": "msg_3Mp9x2Fn8kLpQ4R6",
      "type": "user",
      "content": "What was the revenue growth percentage in Q3?",
      "timestamp": "2025-09-20T15:05:30Z"
    },
    {
      "message_id": "msg_4Nq0x3Gn9mLpR5S7",
      "type": "assistant",

```

```
      "content": "Based on the Q3 Financial Report, the revenue growth was  
23%...",  
      "confidence": 0.92,  
      "timestamp": "2025-09-20T15:05:31Z"  
    }  
  ],  
  "total_messages": 2  
}
```

Error Handling

The DocChat API uses standard HTTP status codes and provides detailed error information.

HTTP Status Codes

Code	Description	Common Causes
200	Success	Request completed successfully
201	Created	Resource created successfully
400	Bad Request	Invalid request format or parameters
401	Unauthorized	Invalid or missing API key
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn't exist
413	Payload Too Large	File exceeds size limit (10MB)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server-side issue
503	Service Unavailable	Temporary service outage

Error Response Format

```
{  
  "error": {  
    "code": "invalid_document_format",  
    "message": "The uploaded file format is not supported. Please use PDF,  
DOCX, or TXT files.",  
    "details": {  
      "supported_formats": ["pdf", "docx", "txt"],  
      "received_format": "xlsx"  
    },  
    "request_id": "req_7Ks4x6Ln2pMpN8Q9"  
  }  
}
```

Common Error Codes

Authentication Errors (401):

- `invalid_api_key`: API key is invalid or expired
- `missing_authorization`: Authorization header is missing

Request Errors (400):

- `invalid_document_format`: Unsupported file type
- `document_too_large`: File exceeds 10MB limit
- `missing_required_field`: Required parameter is missing
- `invalid_json`: Request body is not valid JSON

Rate Limiting (429):

- `rate_limit_exceeded`: Too many requests per minute
 - `quota_exceeded`: Monthly usage quota exceeded
-

Rate Limiting & Security

Rate Limits

Standard Plan:

- 100 requests per minute per API key
- 10,000 requests per month
- 5 concurrent document processing jobs

Pro Plan:

- 500 requests per minute per API key
- 100,000 requests per month
- 25 concurrent processing jobs

Enterprise Plan:

- Custom rate limits
- Dedicated processing resources

Rate Limit Headers

X-RateLimit-Limit: 100
X-RateLimit-Remaining: 87
X-RateLimit-Reset: 1695220800
Retry-After: 45

Security Best Practices

API Key Management:

- Rotate API keys regularly (quarterly recommended)
- Use environment variables, never hardcode keys
- Implement key rotation in production systems
- Monitor API key usage in dashboard

Request Security:

- Always use HTTPS endpoints
- Validate file uploads before sending
- Implement request timeouts (recommended: 30 seconds)
- Log API responses for debugging (exclude sensitive data)

IP Whitelisting (Enterprise): Contact support to restrict API access to specific IP ranges.

SDK Installation

Python SDK

Installation:

```
pip install docchat-python
```

Quick Start:

```
from docchat import DocChatClient

# Initialize client
client = DocChatClient(api_key="dc_live_1234567890abcdef")

# Upload document
with open("document.pdf", "rb") as file:
    document = client.documents.upload(
        file=file,
        name="My Document",
        tags=["important"]
    )

print(f"Document uploaded: {document.document_id}")

# Create chat session
session = client.chat.create_session(
    document_ids=[document.document_id],
    session_name="Analysis Session"
)

# Send message
```



```

response = client.chat.send_message(
    session_id=session.session_id,
    message="Summarize the key points"
)

print(f"Response: {response.response}")

```

JavaScript SDK (Node.js)

Installation:

```
npm install @docchat/javascript-sdk
```

Quick Start:

```

const { DocChatClient } = require('@docchat/javascript-sdk');

// Initialize client
const client = new DocChatClient({
    apiKey: 'dc_live_1234567890abcdef'
});

async function main() {
    // Upload document
    const fs = require('fs');
    const fileStream = fs.createReadStream('document.pdf');

    const document = await client.documents.upload({
        file: fileStream,
        name: 'My Document',
        tags: ['important']
    });

    console.log(`Document uploaded: ${document.document_id}`);

    // Create chat session
    const session = await client.chat.createSession({
        documentIds: [document.document_id],
        sessionName: 'Analysis Session'
    });

    // Send message
    const response = await client.chat.sendMessage({
        sessionId: session.session_id,
        message: 'Summarize the key points'
    });

    console.log(`Response: ${response.response}`);
}

main().catch(console.error);

```

Browser JavaScript (ES6 Modules)

CDN Installation:

```
<script type="module">
  import { DocChatClient } from
  'https://cdn.docchat.com/sdk/v2.1/docchat.esm.js';

  const client = new DocChatClient({
    apiKey: 'dc_test_1234567890abcdef' // Use test key for client-side
  });
</script>
```

Webhooks

Configure webhooks to receive real-time notifications about document processing and chat events.

Webhook Configuration

Endpoint: POST /webhooks

Request Body:

```
{
  "url": "https://your-app.com/webhooks/docchat",
  "events": [
    "document.processing_completed",
    "document.processing_failed",
    "chat.session_created"
  ],
  "secret": "your_webhook_secret_key",
  "active": true
}
```

Response (201 Created):

```
{
  "webhook_id": "wh_5Tr8x1Mn4qPpK6S2",
  "url": "https://your-app.com/webhooks/docchat",
  "events": ["document.processing_completed"],
  "created_at": "2025-09-20T16:00:00Z",
  "active": true
}
```

Webhook Events

Document Events:

- document.processing_started: Document processing began
- document.processing_completed: Document successfully processed

- `document.processing_failed`: Document processing failed

Chat Events:

- `chat.session_created`: New chat session created
- `chat.message_received`: New message in session

Webhook Payload Example

```
{
  "event_id": "evt_9Wx3x5Pn7rQpM4T8",
  "event_type": "document.processing_completed",
  "created_at": "2025-09-20T14:34:18Z",
  "data": {
    "document_id": "doc_2Nq8x7Hn9mKpL3R5",
    "name": "Q3_Financial_Report.pdf",
    "status": "completed",
    "processing_time_ms": 236000,
    "analysis_results": {
      "confidence_score": 0.94,
      "entities_extracted": 156
    }
  }
}
```

Webhook Security

Signature Verification (Python):

```
import hmac
import hashlib

def verify_webhook_signature(payload, signature, secret):
    expected_signature = hmac.new(
        secret.encode('utf-8'),
        payload.encode('utf-8'),
        hashlib.sha256
    ).hexdigest()

    return hmac.compare_digest(
        f"sha256={expected_signature}",
        signature
    )

# Usage in your webhook handler
signature = request.headers.get('X-DocChat-Signature')
if verify_webhook_signature(request.body, signature, webhook_secret):
    # Process webhook
    pass
else:
    # Invalid signature
    return 401
```

Code Examples

Complete Document Processing Workflow (Python)

```
import time
from docchat import DocChatClient

def process_document_and_chat(file_path, question):
    client = DocChatClient(api_key="dc_live_1234567890abcdef")

    # Step 1: Upload document
    with open(file_path, "rb") as file:
        document = client.documents.upload(
            file=file,
            name=file_path.split('/')[-1],
            processing_mode="comprehensive"
        )

    print(f"Uploaded document: {document.document_id}")

    # Step 2: Wait for processing to complete
    while True:
        status = client.documents.get(document.document_id)
        if status.status == "completed":
            break
        elif status.status == "failed":
            raise Exception("Document processing failed")

        print(f"Processing... {status.processing_progress}%")
        time.sleep(5)

    # Step 3: Create chat session
    session = client.chat.create_session(
        document_ids=[document.document_id],
        session_name="Automated Analysis"
    )

    # Step 4: Ask question
    response = client.chat.send_message(
        session_id=session.session_id,
        message=question,
        include_sources=True
    )

    return {
        "answer": response.response,
        "confidence": response.confidence,
        "sources": response.sources
    }

# Usage
result = process_document_and_chat(
    "financial_report.pdf",
    "What are the main revenue sources?"
)
```

```
print(f"Answer: {result['answer']}")
```

Async Document Processing (JavaScript)

```
const { DocChatClient } = require('@docchat/javascript-sdk');

class DocumentProcessor {
  constructor(apiKey) {
    this.client = new DocChatClient({ apiKey });
  }

  async uploadAndProcess(filePath, onProgress) {
    const fs = require('fs');

    // Upload document
    const document = await this.client.documents.upload({
      file: fs.createReadStream(filePath),
      name: filePath.split('/').pop(),
      processing_mode: 'standard'
    });

    // Poll for completion
    return new Promise((resolve, reject) => {
      const checkStatus = async () => {
        try {
          const status = await
this.client.documents.get(document.document_id);

          if (status.status === 'completed') {
            resolve(status);
          } else if (status.status === 'failed') {
            reject(new Error('Processing failed'));
          } else {
            onProgress && onProgress(status.processing_progress);
            setTimeout(checkStatus, 2000);
          }
        } catch (error) {
          reject(error);
        }
      };

      checkStatus();
    });
  }

  async createChatSession(documentIds, name) {
    return await this.client.chat.createSession({
      documentIds,
      sessionName: name
    });
  }

  async askQuestion(sessionId, question) {
    return await this.client.chat.sendMessage({
      sessionId,
      message: question,
    });
  }
}
```

```

        includeSourcecss: true
    });
}
}

// Usage
async function main() {
    const processor = new DocumentProcessor('dc_live_1234567890abcdef');

    try {
        // Process document with progress updates
        const document = await processor.uploadAndProcess(
            'contract.pdf',
            (progress) => console.log(`Processing: ${progress}%`)
        );

        // Create chat session
        const session = await processor.createChatSession(
            [document.document_id],
            'Contract Analysis'
        );

        // Ask multiple questions
        const questions = [
            'What is the contract duration?',
            'What are the payment terms?',
            'Are there any penalties mentioned?'
        ];

        for (const question of questions) {
            const response = await processor.askQuestion(session.session_id,
question);
            console.log(`Q: ${question}`);
            console.log(`A: ${response.response}\n`);
        }

    } catch (error) {
        console.error('Error:', error.message);
    }
}

main();

```

Support & Resources

Documentation: <https://docs.docchat.com>

API Status: <https://status.docchat.com>

Support: support@docchat.com

Community: <https://community.docchat.com>

Need Help?

- Check our [FAQ](#)
- Browse [code examples](#)
- Join our [Discord community](#)

Last updated: September 20, 2025 | API Version: 2.1.3